# CSE 221: Algorithms

Chapter 1 — Asymptotic Analysis

Prepared by Ariyan Hossain [AYO]

## 1 RAM Model

The **RAM (Random Access Machine) model** is a theoretical model for analyzing algorithms. It simplifies how a computer works so we can focus on the efficiency of algorithms (e.g., time and space complexity) without getting distracted by hardware details. The model assumes a single processor and instructions execute one after another (no concurrency). It also assumes that each instruction (operations, data access, data movement) takes the same amount of time as any other instruction.

The RAM model contains instructions commonly found in real computers: *Arithmetic* (add, subtract, multiply), *Data Movement* (assign), *Control Flow* (return), and *Comparison* (logical operators), all taking constant time.

Example: Counting primitive operations

`temp = A[0]` $\rightarrow 2$ (data access and assignment)

`if temp > 10:` $\rightarrow 1$ (comparison)

`return temp` $\rightarrow 1$ (control flow)

**Total Operations** $= 4$

To compare algorithms, we want objective measures:

- **Execution time?** Not good because it depends on the computer.
- **Number of statements executed?** Not good because it depends on the language and coding style.
- **Ideal approach:** Express running time as a function of input size $n$ (i.e., $T(n)$) and compare growth rates.

### 1.1 Running time

The **running time** of an algorithm is the "amount of time" it takes to finish for an input. In algorithm analysis, "time" means the **number of steps (primitive operations)**, not wall-clock time.

Primitive operations include: basic math $(+, -, \times, /)$, comparisons (e.g., $x > y$), loops (`while`, `for`), and assignments (e.g., $x = 5$).

The running time grows with the **input size** (e.g., array length, number of vertices/edges in a graph, etc.). We assume each line $k$ takes constant time $c_k$. The running time, denoted as $T(n)$, is the sum of the costs of executed statements.

We usually focus on the **worst-case** running time ( the maximum number of steps taken on any instance of size $n$), because:

- It gives an upper bound (a guarantee). If you know it, then you have a guarantee that the algorithm never takes any longer.
- In many tasks (e.g., searching), the worst case occurs often (e.g., when the item is missing).

# 2 Order of growth

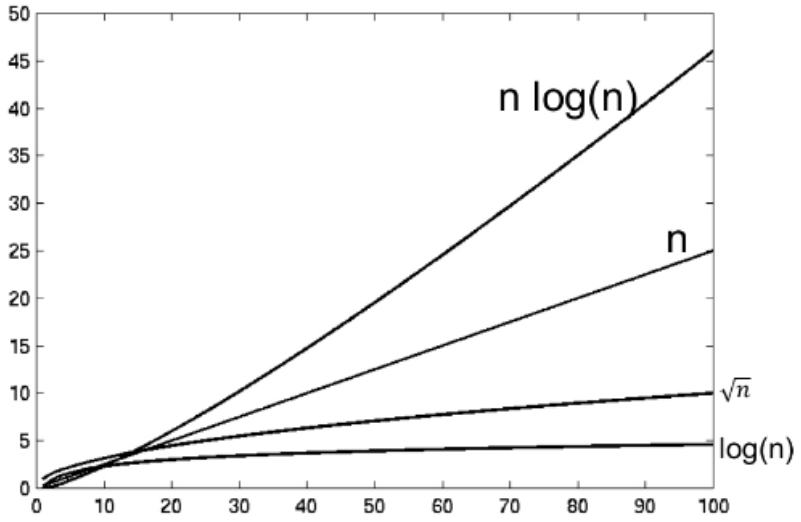
Running time can be viewed as analogous to a mathematical function $f(x)$. For example, the running time of an algorithm can be expressed as $T(n) = 5n^2 + 100n + 1000$, where $n$ represents the input size and $T(n)$ gives the number of operations performed as a function of $n$. We consider only the **leading term** of a running time formula (e.g., $an^2$) since lower-order terms become insignificant for large $n$. We also ignore leading constants because constant factors are less significant than growth rate.

## 2.1 Typical running time functions

- 1 (constant): instructions executed once or a few times
- $\log n$ (logarithmic): repeatedly reduce the problem size by a constant fraction
- $n$ (linear): constant work per input element
- $n \log n$ (superlinear): divide into subproblems, solve independently, combine
- $n^2$ (quadratic): process all pairs (double nested loops)
- $n^3$ (cubic): process triples (triple nested loops)
- $n^k$ (polynomial)
- $c^n$ (exponential): rarely practical for large $n$

| $n$ | 1 | $\log n$ | $n$ | $n \log n$ | $n^2$ | $n^3$ | $2^n$ |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 0.00 | 1 | 0 | 1 | 1 | 2 |
| 10 | 1 | 3.32 | 10 | 33 | 100 | 1,000 | 1024 |
| 100 | 1 | 6.64 | 100 | 664 | 10,000 | 1,000,000 | $1.2 \times 10^{30}$ |
| 1000 | 1 | 9.97 | 1000 | 9970 | 1,000,000 | $10^9$ | $1.1 \times 10^{301}$ |

Example values showing how different growth rates scale with $n$ .




Example plot comparing growth of $\log n$, $\sqrt{n}$, $n$, and $n \log n$

### 2.2 Comparison of orders

A common ordering (from slowest-growing to fastest-growing) is:

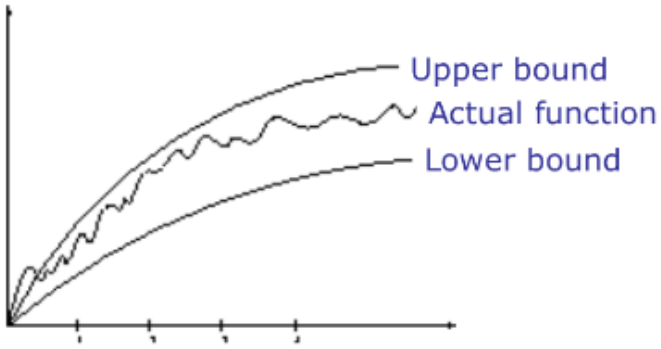
$$\log(\log n) < \log n < \sqrt{n} < n < n \log n < n^{3/2} < n^2 < n^2 \log n < n^3 < 2^n < e^{n+1} < n! < n^n$$

## 3 Asymptotic analysis

Asymptotic analysis studies functions as $n \to \infty$ and helps approximate running time when exact counting is too hard. Best case depends on input; average case can be difficult. So we often use worst-case analysis, which is easier and typically close to practical performance.

**Strategy**

Find a function (an equation) that, for large $n$, is an upper bound to the actual function.




Upper bound, actual function, and lower bound.

### 3.1 Big-Oh ($\mathcal{O}$): asymptotic upper bound

We say $f(n)$ is in the order of $g(n)$, written $f(n) = \mathcal{O}(g(n))$, if there exist a positive constant $c > 0$ and and a positive integer $n_0 > 0$ such that

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

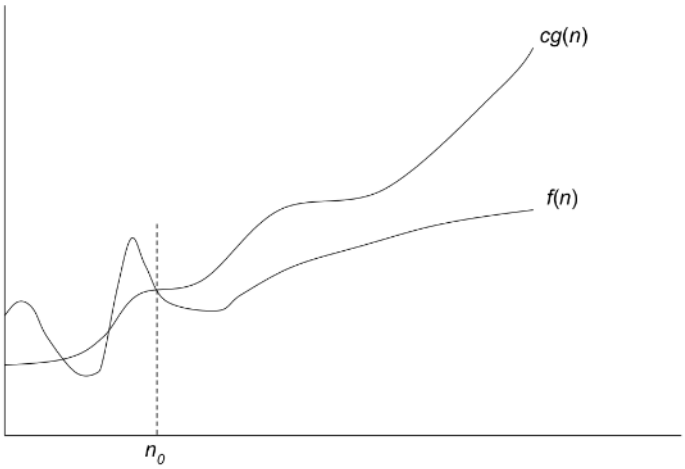



Illustration of an asymptotic upper bound: for $n \geq n_0$, $f(n) \leq c\, g(n)$

**Worked examples (Big-Oh)**

Prove that $2n + 3$ is $\mathcal{O}(n)$

$$2n + 3 \leq 2n + 3$$
$$2n + 3 \leq 2n + 3n$$
$$2n + 3 \leq 5n$$

By the definition of Big-$\mathcal{O}$, choose $c = 5$ and $n_0 = 1$. Therefore, $2n + 3$ is $\mathcal{O}(n)$ since $2n + 3 \leq 5n$ for $n \geq 1$.

Prove that $2n + 3$ is $\mathcal{O}(n^2)$

$$2n + 3 \leq 2n + 3$$
$$2n + 3 \leq 2n^2 + 3n^2$$
$$2n + 3 \leq 5n^2$$

By the definition of Big-$\mathcal{O}$, choose $c = 5$ and $n_0 = 1$. Therefore, $2n + 3$ is $\mathcal{O}(n^2)$ since $2n + 3 \leq 5n^2$ for $n \geq 1$.

Prove that $2^{n+2}$ is $\mathcal{O}(2^n)$

$$2^{n+2} \leq 2^{n+2}$$
$$2^{n+2} \leq (2^n)(2^2)$$
$$2^{n+2} \leq 4 \cdot 2^n$$

By the definition of Big-$\mathcal{O}$, choose $c = 4$ and $n_0 = 1$. Therefore, $2^{n+2}$ is $\mathcal{O}(2^n)$ since $2^{n+2} \leq 4 \cdot 2^n$ for $n \geq 1$.

Prove that $5n^2 + 3n \log n + 2n + 5$ is $\mathcal{O}(n^2)$

$$5n^2 + 3n \log n + 2n + 5 \leq 5n^2 + 3n \log n + 2n + 5$$
$$5n^2 + 3n \log n + 2n + 5 \leq 5n^2 + 3n^2 + 2n^2 + 5n^2$$
$$5n^2 + 3n \log n + 2n + 5 \leq 15n^2$$

By the definition of Big-$\mathcal{O}$, choose $c = 15$ and $n_0 = 1$. Therefore, $5n^2 + 3n \log n + 2n + 5$ is $\mathcal{O}(n^2)$ since $5n^2 + 3n \log n + 2n + 5 \leq 15n^2$ for $n \geq 1$.

**Prove that $5^n$ is *not* $\mathcal{O}(4^n)$**

Suppose $5^n$ is $\mathcal{O}(4^n)$. Then there exists a positive constant $c$ and $n_0$ such that

$$5^n \leq c \cdot 4^n \quad \text{for } n \geq n_0$$

Dividing by $4^n$ gives

$$\left(\frac{5}{4}\right)^n \leq c$$
$$1.25^n \leq c$$

But $1.25^n$ grows without bound, so it cannot be bounded by a constant $c$. This contradiction shows $5^n$ is not $\mathcal{O}(4^n)$.

## 3.2 Big-Omega ($\Omega$): asymptotic lower bound

We say $f(n)$ is in the order of $g(n)$, written $f(n) = \Omega(g(n))$, if there exist a positive constant $c > 0$ and and a positive integer $n_0 > 0$ such that

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{for all } n \geq n_0$$

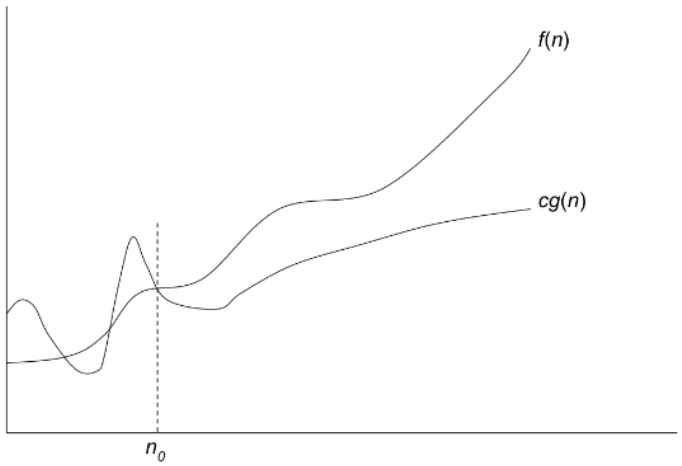



Illustration of an asymptotic lower bound: for $n \geq n_0$, $f(n) \geq c\,g(n)$

**Worked examples (Big-Omega)**

**Prove that $8n^3 + 5n^2 + 7$ is $\Omega(n^3)$**

$$8n^3 + 5n^2 + 7 \geq 8n^3 + 5n^2 + 7$$
$$8n^3 + 5n^2 + 7 \geq 8n^3$$

By the definition of Big-$\Omega$, choose $c = 8$ and $n_0 = 1$. Therefore, $8n^3 + 5n^2 + 7$ is $\Omega(n^3)$ since $8n^3 + 5n^2 + 7 \geq 8n^3$ for $n \geq 1$.

**Prove that $3^n$ is $\Omega(2^n \cdot n^4)$**

Suppose $3^n$ is $\Omega(2^n \cdot n^4)$. Then there exists a positive constant $c$ and $n_0$ such that

$$3^n \geq c \cdot 2^n \cdot n^4 \quad \text{for } n \geq n_0$$

Dividing by $2^n$ gives

$$\left(\frac{3}{2}\right)^n \geq c \cdot n^4$$
$$1.5^n \geq c \cdot n^4$$

An exponential term dominates any polynomial for sufficiently large $n$. By the definition of Big-$\Omega$, choose $c = 1$ and $n_0 = 100$. Therefore, $3^n$ is $\Omega(2^n \cdot n^4)$ since $3^n \geq 2^n \cdot n^4$ for $n \geq 100$.

### 3.3 Big-Theta ($\Theta$): asymptotic tight bound

We say $f(n)$ is $\Theta(g(n))$ if there exist positive constants $c_1 > 0$, $c_2 > 0$, and a positive integter $n_0 > 0$ such that

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0$$

Equivalently, $f(n) = \Theta(g(n))$ if $f(n) = \mathcal{O}(g(n))$ *and* $f(n) = \Omega(g(n))$.

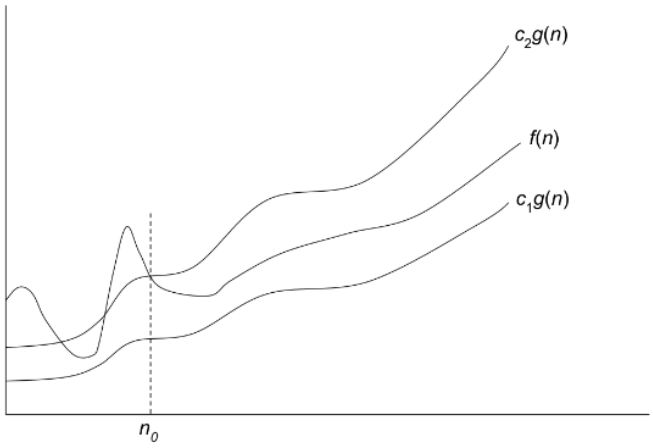



Illustration of a tight bound: for $n \geq n_0$, $c_1 g(n) \leq f(n) \leq c_2 g(n)$

**Worked examples (Big-Theta)**

Prove that $20n^3 + 7n + 1000$ is $\Theta(n^3)$

**Upper bound:**

$$20n^3 + 7n + 1000 \leq 20n^3 + 7n + 1000$$
$$20n^3 + 7n + 1000 \leq 20n^3 + 7n^3 + 1000n^3$$
$$20n^3 + 7n + 1000 \leq 1027n^3$$

By the definition of Big-$\mathcal{O}$, choose $c = 1027$ and $n_0 = 1$. Therefore, $20n^3 + 7n + 1000$ is $\mathcal{O}(n^3)$ since $20n^3 + 7n + 1000 \leq 1027n^3$ for $n \geq 1$.

**Lower bound:**

$$20n^3 + 7n + 1000 \geq 20n^3 + 7n + 1000$$
$$20n^3 + 7n + 1000 \geq 20n^3$$

By the definition of Big-$\Omega$, choose $c = 20$ and $n_0 = 1$. Therefore, $20n^3 + 7n + 1000$ is $\Omega(n^3)$ since $20n^3 + 7n + 1000 \geq 20n^3$ for $n \geq 1$.

Since both bounds hold, $20n^3 + 7n + 1000$ is $\Theta(n^3)$.

Prove that $n^2 + 15n - 3$ is $\Theta(n^2)$

**Upper bound:**

$$n^2 + 15n - 3 \leq n^2 + 15n - 3$$
$$n^2 + 15n - 3 \leq n^2 + 15n^2$$
$$n^2 + 15n - 3 \leq 16n^2$$

By the definition of Big-$\mathcal{O}$, choose $c = 16$ and $n_0 = 1$. Therefore, $n^2 + 15n - 3$ is $\mathcal{O}(n^2)$ since $n^2 + 15n - 3 \leq 16n^2$ for $n \geq 1$.

**Lower bound:**,

$$n^2 + 15n - 3 \geq n^2 + 15n - 3$$
$$n^2 + 15n - 3 \geq n^2$$

By the definition of Big-$\Omega$, choose $c = 1$ and $n_0 = 1$. Therefore, $n^2 + 15n - 3$ is $\Omega(n^2)$ since $n^2 + 15n - 3 \geq n^2$ for $n \geq 1$.

Since both bounds hold, $n^2 + 15n - 3$ is $\Theta(n^2)$.